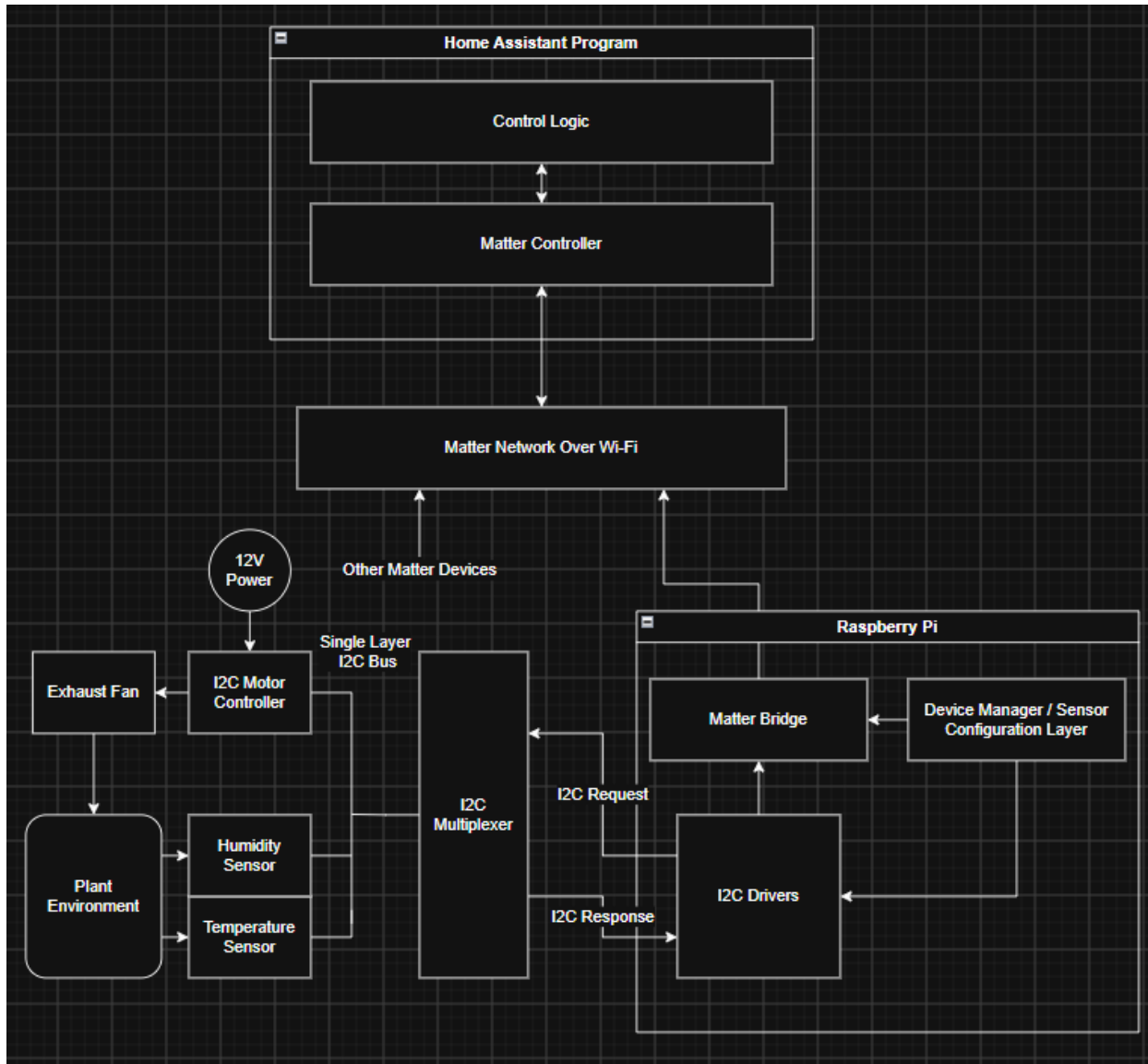


Jacob Kozik

7/20/2026

Luce Labs Candidate Briefing Project

Architecture Overview:



The main purpose of this design is to maximize the modularity and longevity while minimizing the cost and design complexity for the end user. This was accomplished by taking advantage of the matter integration within Home Assistant and the shared bus capability of the I2C protocol. Additionally, I2C multiplexers were used so that sensors can

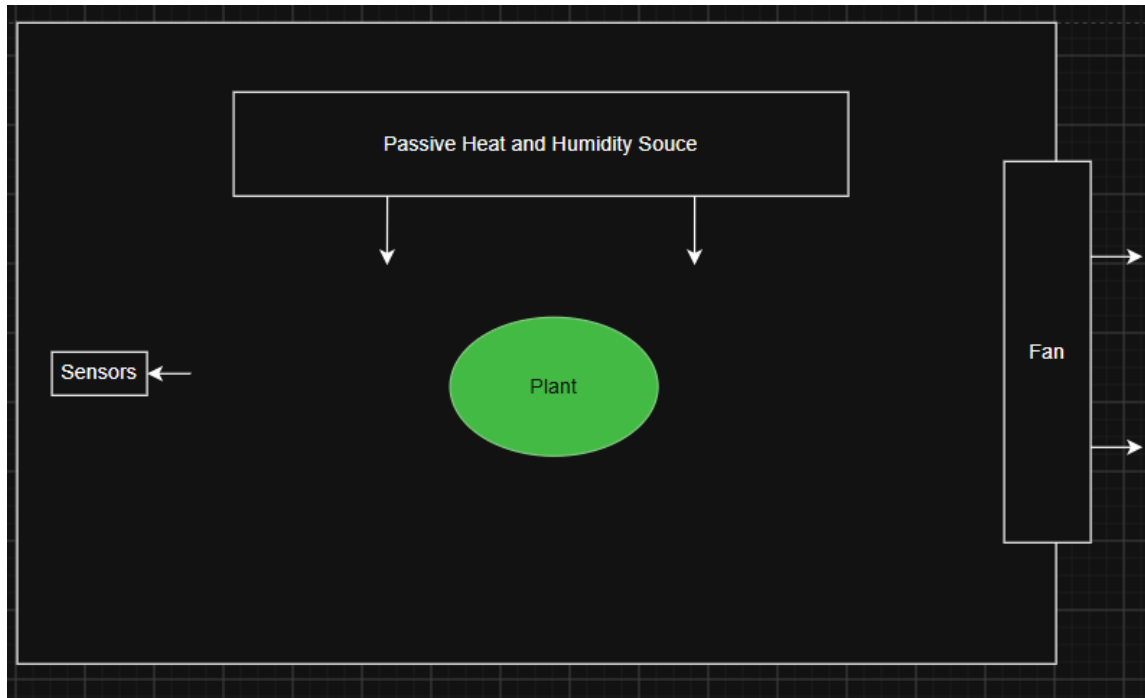
be uniquely identified and reused at every layer in the rack. Most of the complexity in this system is within the raspberry pi, to make the sensors plug and play, the raspberry pi requires a user interface that can 1) detect when a new device is connected and 2) assign the device to an object in the matter network. The Matter Controller, located on the Home Assistant, subscribes to the devices the raspberry pi publishes. Each layer in the rack will be a unique device and will have updated sensor data and actuator controls. The layers will be configured as Matter devices using [Matter SDK](#) which is built in C++. The raspberry pi software may be written completely in C++ or as a mixture of C++ and python, where python will handle the application layer that interfaces with Linux and the user and C++ will be used for Matter integration and hardware interfacing (such as I2C communication).

The benefit of this architecture is that it can allow ease of use for the user to set up a new rack or layer. Ideally the user will plug a sensor into one of the I2C ports located on a layer, the raspberry pi will detect the new I2C address, and the user will select from a list of known devices such as sensors and actuators. Once that selection is made, the raspberry pi will create the matter device of the layer so it can interface with the HA, allowing each layer to be individually controlled. The downside of this architecture is that it is reliant on I2C integration and requires cabling, as opposed to a ZigBee setup. The I2C connectors will likely need protection from humidity so that rust is less likely to form. There is also a lot of complexity in the raspberry pi and its software will probably take the longest amount of time and require additional planning.

There are multiple single point of failures within this model such as the raspberry pi and the main I2C bus connected to it. All signals travel through the raspberry pi and over Wi-Fi and if the raspberry pi fails then the whole rack will stop functioning. The other single point of failure is the HA. If the HA goes down, then all racks will stop functioning. Without changing the hardware, a good work around would be to run the control logic on the raspberry pi and the HA only send parameters for a prebuilt control function. This would allow the racks to function when Wi-Fi goes down or the HA loses power. The downside of this would be that the control function is fixed on the raspberry pi and cannot be modified directly by the HA. A more robust but more complicated system would use the HA as a means of communicating the entire control function to the raspberry pis.

Control Overview:

Below is a simplified physical layout:



The controlled parameters are relative humidity and temperature, and the actuator is an exhaust fan. If there is a base supply of water and heat (perhaps from a mister and light source) then the humidity and heat will rise. The exhaust fan acts like a homogeneous regulator, affecting both humidity and ambient temperature. The controller would implement a threshold-based hysteresis controller, with an additional digital lowpass filter to reduce measurement noise if necessary. Below is the pseudo-code:

```

// Basic Lowpass filter (moving average) if sampling rate is sufficient
temperature = (previous_temperature + current_temperature)/2
relative_humidity = (previous_humidity + current_humidity)/2

// Tuning parameter placeholders
humidity_buffer = 5
temp_buffer = 5
target_temp = 20
target_humidity = 60
short_cycle_delay = 500

// Control logic
current_time = get_time()
state_duration = current_time - last_state_change

if fan_state == on AND (temperature <= target_temp - temp_buffer AND relative_humidity <=
target_humidity - humidity_buffer) AND state_duration >= short_cycle_delay:
    fan_state = off
    last_state_change = current_time

else if fan_state == off AND (temperature > target_temp OR relative_humidity > target_humidity) AND
state_duration >= short_cycle_delay:
    fan_state = on
    last_state_change = current_time

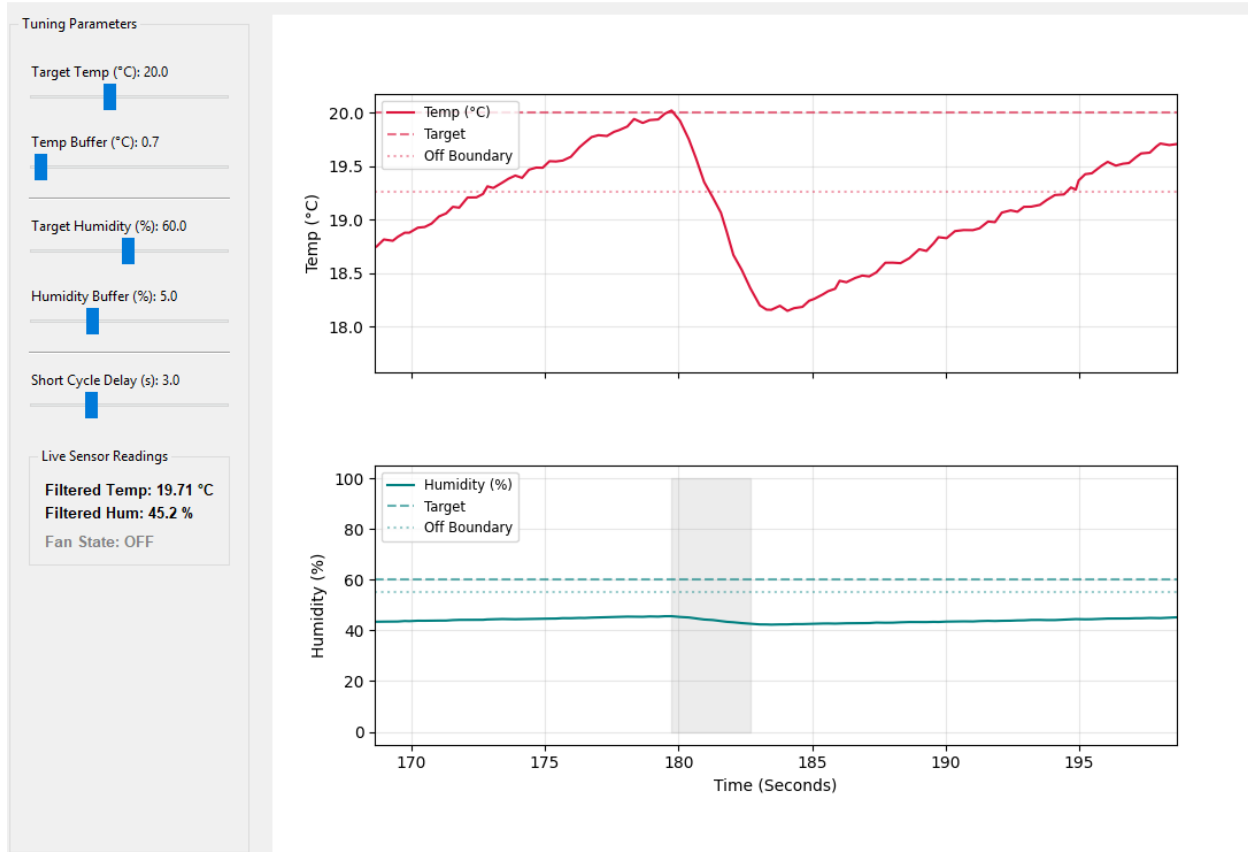
```

This controller essentially cleans the input signals and turns the fan on when either the temperature is high or humidity is high. The fan turns off before reaching the target temperature and humidity. To allow the system to settle into equilibrium, an anti-short cycling delay is added, preventing the actuator state from changing too quickly. Vapor Pressure Deficit is a significant contributor to plant growth and is primarily calculated from temperature and relative humidity. Although the controller, in theory, regulates both parameters, there may be steady state error in one of the parameters. More specifically, it is the one that changes slower, humidity. After running a crude python simulation of the hysteresis controller made from Gemini, the controller introduced a steady state error in the humidity level while the temperature sustained oscillations between the threshold levels.

A possible improvement to the controller without additional hardware would be to use a bidirectional fan. When the temperature is high, but the humidity is low then the fan

will blow colder air into the enclosure while keeping the humidity inside. I did not think of this until after creating the whole design, but I would've preferred this method over the current one.

The python program output is shown below:



Parts Matrix:

Humidity Sensor:

Humidity Sensors	Price	Humidity Accuracy	Temperature Accuracy	Pressure Accuracy
Adafruit Sensirion SHT45	\$12.50	±1.0% RH	±0.1°C	N/A
Adafruit BME280	\$14.95	±3% RH	±1°C	±1 hPa

Most Humidity Sensors do not come as a standalone device so instead of making a custom PCB, it's probably better to buy a combined PCB with temperature and humidity such as the Adafruit Sensirion SHT45. For a less accurate sensor with additional pressure

capability, the Adafruit BME280 can be used. The sensor data is communicated on the same I2C address and accessed via the bytes of the communicated packet.

Exhaust Fan Controller:

Controller	Price	Temperature Accuracy
Adafruit EMC2101	\$5.50	±1°C

The Adafruit EMC2101 requires 12V for the PWM signal but can allow modularity at the cost of an extra 12V wire. It also features a built-in temperature sensor which can be used to better approximate the total temperature of the enclosure. It also benefits the plug-and-play capability since it interfaces with the I2C bus and features a 4-pin PWM connector for different computer fan models.

Exhaust Fans:

Fan	Price	Bearings	Water Resistance
ARCTIC P12 PWM PST CO	\$10.99	Continuous-operation bearing	n/a
Noctua NF-F12 iPPC-2000 IP67 PWM	\$37.33	SSO2	IP67 rating
be quiet! SILENT WINGS 4 120mm PWM	\$24.90	Fluid Dynamic	n/a

All the fans above are compatible with the controller previously mentioned. The best fan for longevity is the Noctua NF-F12 iPPC-2000 IP67 PWM since it is rated for humid environments and can get wet from misting. However, it is 4x the price of the Arctic so there is a trade off.

If I could choose one component to design from scratch it would be the fan motor controller because it currently requires soldering a 12V power supply to the PCB. I would instead make it use a standard 2 pin connector.

Home Assistant:

Gemini converted my simulation and pseudo code into multiple YAML files for Home Assistant. I wasn't able to get Home Assistant running on WSL within VSCode so I am just submitting the results of the prompt within the GitHub repo.

AI Appendix:

Notable Prompts:

- Can you provide a list of 3 best value temperature sensors and 3 humidity sensors that communicate over I2C, they should be durable and be able to function for very long durations. The sensors should be a standalone fully made product that uses a 4-pin JST-SH connector for power and communication.
- Create a list of 3 best valued exhaust fans that are connected with a 4-pin PWM connector at 12Vpp. The fans should be durable and capable of handling humid environments.

Significant AI Correction:

The original actuator was going to be a humidifier that would be controlled with a Matter smart plug. This was ChatGPT's first suggestion but after diving deeper into the humidifiers that are available, I didn't think that they were worth it as it would make the use of Home Assistant obsolete. The solution I came up with was to use an exhaust fan.

AI Usage:

ChatGPT and Gemini were used heavily in this project for bouncing ideas back and forth and testing architectural ideas before prototyping. I started the project not knowing anything about home assistant, Zigbee, Matter, Thread, etc so I used AI to teach me through asking it questions and running through application scenarios. In general, this is what I usually use AI for. I also leveraged Gemini to implement my controller pseudo code into a simple simulation program in python so I can test my theory of steady state error. The experiment was useful (after some debugging) and it helped me see how my controller would function before implementing. Although I didn't check the math completely, the entire process only took about 15 minutes and the results looked reasonable. Gemini was used for YAML. The final document was written completely by hand.

Time Spent:

1 Hour: talking with ChatGPT about the pdf, discussing what home assistant is and what the protocols are. Also used ChatGPT for brainstorming what parameters can be controlled with a single actuator. Found that you can control temperature and humidity with a humidifier.

1 Hour: researching different system architectures and protocols and evaluating the compatibility with the current system and the desired characteristics such as plug and play

1 Hour: Prototyped powered USB hub architecture for multiple layered racks and found that although simpler than lower-level designs, it would require a lot of USB cables. For an 8 layered rack with 3 sensors, there would be 24 USB cables going into one hub and the hub alone would cost between 20-40 dollars.

Problems:

- No actuation integration
- USB is great for plug and play but wiring can get expensive and ugly

1.5 Hours: prototyped I2C sensor architecture with a local raspberry pi, taking advantage of the shared wiring bus. Wireless communication with the Home Assistant is through MQTT with the Home Assistant hosting an MQTT broker and interfacing with a built-in interface in the Home Assistant software.

Problems:

- I2C sensors will repeat addresses
- Wireless communication would use both Matter and MQTT layers, making the HA integration more complicated
- I2C sensors require configuration on the raspberry pi when they are first connected
 - o This may be achieved with a discovery software local on the raspberry pi

1 Hour: refined the I2C architecture by discovering I2C multiplexers, making it possible to control 8 racks, each with multiple sensors and actuators. The wireless communication layer on the raspberry pi was switched to use a Matter Bridge; this will require device configuration for each sensor. The actuator is a basic preconfigured humidifier controlled by a Matter smart plug. The matter smart plug and the Matter bridge on the raspberry pi are configured in the home assistant settings.

Problems:

- Sensor configuration now involves matter integration
- I2C connectors may get rusty

1 Hour: Searched for compatible temperature and humidity sensors, as well as humidifiers. However, after searching for a long time for a humidifier, it was very difficult to find one that is suitable for the project. I did find an I2C compatible sensor that senses humidity, temperature, and pressure.

30 mins: Brainstorming with ChatGPT about other closely coupled parameters and alternatives to a humidifier. I decided on an exhaust fan as the actuator, it is not perfect by any means, but it can work. The exhaust fan can be controlled by the I2C bus at the cost of an additional 12V power line.

1 Hour: Researched part selection matrix for the fans and controller, created the control algorithm, and added more documentation.

1 Hour: Researched Matter bridge implementation on the raspberry pi using ChatGPT. ChatGPT kept sending conflicting coding examples so learning through the documentation is better.

1 Hour: Created the control pseudo code and control section.

3 Hours: Completing deliverables and testing

2 Hours: Finishing the documentation

1 Hour: Gemini implemented my controller function into YAML files for HA to test on Docker. Switched to run Home Assistant in a virtual environment within VSCode. I couldn't get home assistant working so I am submitting untested YAML files that were created by Gemini.